YATRIO

7-Team Detailed Task Assignment

A practical handoff document for 21 members — every team owns Frontend + Backend + Database + SQL + Integration/Testing.

Recommended team size: 3 members minimum (Frontend / Backend / Database). If a team has 4 members, the fourth owns Integration + Testing.

Team	Module	Member 1	Member 2	Member 3	Member 4 (if available)
1	Authentication & Users	Frontend	Backend	Database	Integration/Security
2	Railway Search	Frontend	Backend/API	Database	Integration/Testing
3	Train Details & Availability	Frontend	Backend	Database/Redis	Integration/Testing
4	Booking & Passengers	Frontend	Backend	Database/Transactions	Integration/Testing
5	Payment	Frontend	Backend/Razorpay	Database	Integration/Security Testing
6	PNR & Live Tracking	Frontend/Map	Backend/API	Database/Redis	Integration/Testing
7	Admin & Analytics	Admin Frontend	Admin Backend	DB/Analytics	DevOps/QA

TEAM 1 — AUTHENTICATION & USER MANAGEMENT

Team Goal: Own the identity layer: users must be able to register, authenticate, manage profiles and receive the correct role permissions.

Area	Tasks to Complete	Expected Output / Acceptance
FRONTEND	Build Register, Login, Logout, Forgot Password, Reset Password and Profile pages. Add form validation, loading/error states, protected routes, auth state store and role-based navigation.	Register/Login/Profile flow works on desktop and mobile.
BACKEND	Create auth/user controllers, services and routes. Implement password hashing, JWT/access-refresh strategy or chosen session strategy, authentication middleware, role middleware, validation and centralized auth errors.	POST /auth/register, /login, /logout, /forgot-password, /reset-password; GET/PUT /users/me; PUT /users/change-password.
DATABASE	Create users, roles, user_roles, user_sessions and password_resets. Normalize profile data where required. Add PK/FK/UNIQUE constraints and useful indexes.	ER relationship diagram + Sequelize models + migration files + seed data.
PROCEDURES / TRIGGERS	Create register/update profile procedures if the team uses procedures. Add audit trigger for important user changes and optional session cleanup logic.	register_user(), update_user_profile(); user_audit trigger.
SECURITY / OTHER	Password policy, input sanitization, rate limiting for login/reset endpoints, token expiry, authorization tests. Never store passwords in plain text.	Security checklist and negative test cases.
TESTING / HANDOFF	Unit/API tests for registration, duplicate email, wrong password, expired token, unauthorized role and password reset. Document API request/response formats.	Complete Auth API + Postman collection + README.

Team Definition of Done

• Frontend screens are responsive and connected to the real team API.

• Backend routes use validation, consistent error responses and authentication/authorization where required.

• Database changes are version-controlled through migrations/SQL files.

• At least one meaningful stored procedure, trigger, view or indexing task is completed where listed.

• API endpoints are documented with request/response examples.

• Happy-path and major failure/edge cases are tested.

• Team branch can be merged without manual changes to another team's code/database.

• README explains setup, environment variables, endpoints and known dependencies.

TEAM 2 — RAILWAY SEARCH & STATION DISCOVERY

Team Goal: Build the discovery engine that lets a user find trains between two stations for a selected date.

Area	Tasks to Complete	Expected Output / Acceptance
FRONTEND	Build station autocomplete, From/To selection, date picker, swap stations, search button, filters and train result cards. Handle empty/error/loading states.	Search → Results page with reusable train-card component.
BACKEND	Build station and train search controllers/services. Normalize external railway API data into Yatrio's response format. Validate station codes and dates.	GET /stations, /stations/search, /trains/search, /trains/between, /trains/:trainNo.
DATABASE	Create stations, trains, train_routes, train_running_days and train_classes. Store railway master data in normalized form. Add indexes on station code, train number and route lookup.	Migration + seed/master-data import script.
PROCEDURES / TRIGGERS	Create station search and between-station train search procedures where appropriate. Add audit trigger for master-data changes.	search_stations(), search_trains_between_stations().
API / CACHE	Define adapter for the selected railway API. Decide what is fetched live versus stored as master data. Cache high-frequency station/train lookup data if useful.	API adapter + documented cache policy.
TESTING / HANDOFF	Test same-station search, invalid station, past date, no trains, multiple trains, pagination/filtering and API failure.	Postman collection + search response contract + sample dataset.

Team Definition of Done

• Frontend screens are responsive and connected to the real team API.

• Backend routes use validation, consistent error responses and authentication/authorization where required.

• Database changes are version-controlled through migrations/SQL files.

• At least one meaningful stored procedure, trigger, view or indexing task is completed where listed.

• API endpoints are documented with request/response examples.

• Happy-path and major failure/edge cases are tested.

• Team branch can be merged without manual changes to another team's code/database.

• README explains setup, environment variables, endpoints and known dependencies.

TEAM 3 — TRAIN DETAILS & AVAILABILITY

Team Goal: Own everything shown after a user selects a train: schedule, route, class, fare and demo seat availability.

Area	Tasks to Complete	Expected Output / Acceptance
FRONTEND	Build train-details page, route timeline, boarding/dropping station selector, class cards, quota selector, fare display and availability indicators.	Train Details → Availability UI.
BACKEND	Create train detail, schedule, class, fare and availability services. Integrate live/read-only railway data where applicable and merge with Yatrio demo availability data.	GET /trains/:trainNo/details, /schedule, /availability, /fare, /classes.
DATABASE	Create train_schedules, seat_classes, coaches, coach_seats, quotas, fares and availability_snapshots. Define relationships and constraints carefully.	Availability model + migrations + realistic seed data.
PROCEDURES / TRIGGERS	Implement get_train_schedule(), get_train_availability(), calculate_fare() and get_available_seats() where procedures are required. Maintain fare/availability history if needed.	Procedure scripts + trigger scripts + test calls.
REDIS	Cache schedule, fare and availability responses. Define keys and TTLs, e.g. train:{trainNo}:schedule and train:{trainNo}:availability:{date}. Invalidate/refresh cache when demo data changes.	Redis key/TTL documentation + cache service.
TRANSACTIONS / TESTING	Prevent inconsistent availability reads. Test class unavailable, WL/RAC demo state, stale cache, API timeout and invalid train number.	Availability test suite + handoff contract for Team 4.

Team Definition of Done

• Frontend screens are responsive and connected to the real team API.

• Backend routes use validation, consistent error responses and authentication/authorization where required.

• Database changes are version-controlled through migrations/SQL files.

• At least one meaningful stored procedure, trigger, view or indexing task is completed where listed.

• API endpoints are documented with request/response examples.

• Happy-path and major failure/edge cases are tested.

• Team branch can be merged without manual changes to another team's code/database.

• README explains setup, environment variables, endpoints and known dependencies.

TEAM 4 — BOOKING & PASSENGER MANAGEMENT

Team Goal: Own the core booking transaction from passenger entry through reservation and cancellation.

Area	Tasks to Complete	Expected Output / Acceptance
FRONTEND	Build passenger form, add/remove passenger, age/gender/berth preference, contact information, booking summary, fare breakup and cancellation UI.	Passenger → Review → Booking confirmation flow.
BACKEND	Create booking service, passenger service, seat reservation and cancellation service. Validate ownership before allowing booking/cancellation changes.	POST/GET/PUT/DELETE /bookings; passenger endpoints; /bookings/:id/cancel.
DATABASE	Create bookings, booking_passengers, booking_segments, seat_allocations, booking_status and cancellations. Add FK constraints and indexes on user/PNR/status/date.	Booking ERD + migrations + seed/demo booking data.
PROCEDURES / TRIGGERS	Implement create_booking(), reserve_seats(), cancel_booking() and get_booking_details(). Synchronize seat/availability state with triggers only where safe and documented.	Procedure + trigger scripts with rollback tests.
TRANSACTIONS / CONCURRENCY	Use DB transactions for reservation. Lock/check availability before allocation. Ensure failed booking rolls back all related inserts/updates.	BEGIN → validate → reserve → create booking → COMMIT; failure → ROLLBACK.
TESTING / HANDOFF	Test double booking, two simultaneous users, cancellation, invalid passenger data, unavailable seat and partial failure. Coordinate closely with Team 5 payment flow.	Booking API contract + integration tests + sequence diagram.

Team Definition of Done

• Frontend screens are responsive and connected to the real team API.

• Backend routes use validation, consistent error responses and authentication/authorization where required.

• Database changes are version-controlled through migrations/SQL files.

• At least one meaningful stored procedure, trigger, view or indexing task is completed where listed.

• API endpoints are documented with request/response examples.

• Happy-path and major failure/edge cases are tested.

• Team branch can be merged without manual changes to another team's code/database.

• README explains setup, environment variables, endpoints and known dependencies.

TEAM 5 — PAYMENT & TRANSACTIONS

Team Goal: Own the demo payment lifecycle using Razorpay Test Mode and connect payment state safely to booking state.

Area	Tasks to Complete	Expected Output / Acceptance
FRONTEND	Build payment summary, Razorpay checkout integration, processing state, success/failure/pending pages, retry and transaction history.	Booking → Payment → Result flow.
BACKEND	Create payment order, verify gateway signature, fetch payment status, handle retry/refund and protect payment endpoints. Never trust amount/status directly from the client.	POST /payment/create-order, /verify, /refund; GET payment/history/status.
DATABASE	Create payments, payment_transactions, refunds and optionally payment_methods. Store gateway order/payment/refund IDs, amount, currency, status and timestamps.	Payment schema + constraints preventing duplicate transaction IDs.
PROCEDURES / TRIGGERS	Create payment record, confirm payment, refund and payment-history procedures if required. Trigger/event should update booking state only through a controlled service/transaction boundary.	create_payment_record(), confirm_payment(), process_refund().
SECURITY / INTEGRATION	Verify Razorpay signature server-side. Never expose secret keys to React. Coordinate booking reservation timeout/payment expiry with Team 4.	Environment variable checklist + payment sequence diagram.
TESTING / HANDOFF	Test success, failure, duplicate callback/verification, tampered amount, expired order, retry and refund. Use Razorpay test credentials only.	Payment API contract + Postman tests + Team 4 integration guide.

Team Definition of Done

• Frontend screens are responsive and connected to the real team API.

• Backend routes use validation, consistent error responses and authentication/authorization where required.

• Database changes are version-controlled through migrations/SQL files.

• At least one meaningful stored procedure, trigger, view or indexing task is completed where listed.

• API endpoints are documented with request/response examples.

• Happy-path and major failure/edge cases are tested.

• Team branch can be merged without manual changes to another team's code/database.

• README explains setup, environment variables, endpoints and known dependencies.

TEAM 6 — PNR + LIVE TRAIN STATUS + MAP

Team Goal: Create Yatrio's live journey experience: PNR information, train running status, route and current location on a map.

Area	Tasks to Complete	Expected Output / Acceptance
FRONTEND	Build PNR form/result, passenger status cards, live train status, delay indicator, route timeline and map marker/polyline UI.	PNR → Live Status → Map experience.
BACKEND	Create PNR lookup and live-status adapters. Normalize external API responses and expose stable Yatrio endpoints. Handle unavailable/stale live data.	GET /pnr/:pnr; /trains/:trainNo/live, /route, /location.
DATABASE	Create pnr_records, pnr_passengers, train_live_status, train_locations and train_status_history. Store timestamps so stale data can be detected.	Migrations + indexes on PNR, train and timestamps.
PROCEDURES / TRIGGERS	Create get_pnr_details(), get_latest_train_location() and status-history logic. Avoid excessive DB writes for rapidly changing location data.	Procedure/trigger scripts + history cleanup strategy.
REDIS / MAP	Cache live status and PNR responses with short TTLs. Define live:train:{trainNo}, pnr:{pnr}, train:{trainNo}:position. Integrate map provider and route coordinates.	Redis service + map data contract.
TESTING / HANDOFF	Test invalid PNR, stale live data, API unavailable, delayed train, missing coordinates and rapid repeated requests. Verify map route/marker behavior.	Live-status API contract + map integration notes.

Team Definition of Done

• Frontend screens are responsive and connected to the real team API.

• Backend routes use validation, consistent error responses and authentication/authorization where required.

• Database changes are version-controlled through migrations/SQL files.

• At least one meaningful stored procedure, trigger, view or indexing task is completed where listed.

• API endpoints are documented with request/response examples.

• Happy-path and major failure/edge cases are tested.

• Team branch can be merged without manual changes to another team's code/database.

• README explains setup, environment variables, endpoints and known dependencies.

TEAM 7 — ADMIN, ANALYTICS & SYSTEM INFRASTRUCTURE

Team Goal: Build the internal control center and own cross-project database quality, reporting and shared infrastructure.

Area	Tasks to Complete	Expected Output / Acceptance
FRONTEND	Build admin login/guard, dashboard, user list, train/master-data view, booking list, payment list, cancellation list, PNR/API monitoring and charts.	Admin dashboard with filters, pagination and key KPIs.
BACKEND	Build admin-only middleware, dashboard endpoints, report endpoints and management endpoints. Add pagination/filtering and centralized admin authorization.	GET /admin/dashboard, /users, /bookings, /payments, /reports, /analytics.
DATABASE	Create audit_logs and optional api_request_logs. Build reporting views such as booking summary, revenue, train occupancy and user activity. Review all teams' indexes/constraints.	Cross-module DB review + reporting schema.
PROCEDURES / TRIGGERS	Create get_dashboard_statistics(), get_daily_revenue(), get_popular_routes(), get_train_occupancy(). Coordinate audit triggers without duplicating business logic.	Reporting procedure/view scripts.
REDIS / DEVOPS	Own shared Redis conventions, rate limiting, cache monitoring and environment/config documentation. Help coordinate Docker/deployment configuration if used.	Redis key policy + environment matrix + deployment checklist.
QA / OPTIMIZATION	Run EXPLAIN on expensive queries, identify missing indexes, test pagination, audit permissions and run end-to-end regression before release.	Performance report + release checklist + cross-team integration sign-off.

Team Definition of Done

• Frontend screens are responsive and connected to the real team API.

• Backend routes use validation, consistent error responses and authentication/authorization where required.

• Database changes are version-controlled through migrations/SQL files.

• At least one meaningful stored procedure, trigger, view or indexing task is completed where listed.

• API endpoints are documented with request/response examples.

• Happy-path and major failure/edge cases are tested.

• Team branch can be merged without manual changes to another team's code/database.

• README explains setup, environment variables, endpoints and known dependencies.

DATABASE — MASTER TABLE LIST

Use this as the team's common database contract. Exact columns can be refined during ERD design, but ownership should remain stable.

T1 — AUTH

Table	Purpose	Owner	Key / Design Note
users	Registered users and profile/account data.	T1	user_id; email/phone UNIQUE
roles	Application roles such as USER and ADMIN.	T1	role_id; role_name UNIQUE
user_roles	Maps users to roles.	T1	user_id + role_id
user_sessions	Active sessions/refresh-token records.	T1	session_id; user_id
password_resets	Temporary reset token records and expiry.	T1	reset_id; user_id

T2 — SEARCH / RAILWAY MASTER

Table	Purpose	Owner	Key / Design Note
stations	Station master: code, name, city/state and location metadata.	T2	station_id; station_code UNIQUE
trains	Train master: number, name, type.	T2	train_id; train_number UNIQUE
train_routes	Ordered stations/stops for each train.	T2	train_id + station_id + sequence
train_running_days	Days on which a train operates.	T2	train_id + day
train_classes	Classes offered by each train.	T2/T3	train_id + class_id

T3 — AVAILABILITY

Table	Purpose	Owner	Key / Design Note
train_schedules	Scheduled arrival/departure and halt information.	T3	train_id + station_id + sequence
seat_classes	Master class definitions such as 1A/2A/3A/CC/SL.	T3	class_id; code UNIQUE
coaches	Coach instances attached to a train/class.	T3	coach_id; train_id
coach_seats	Individual seat/berth definitions in a coach.	T3	seat_id; coach_id
quotas	Quota definitions used in demo availability.	T3	quota_id; code UNIQUE
fares	Fare rules/amounts for class/route/quota/date as required.	T3	fare_id + relevant FKs
availability_snapshots	Date-specific demo availability and inventory snapshot.	T3	train/class/date/quota

T4 — BOOKING

Table	Purpose	Owner	Key / Design Note
bookings	Booking header: user, journey, PNR/reference, fare and status.	T4	booking_id; user_id
booking_passengers	Passenger rows belonging to a booking.	T4	passenger_id; booking_id
booking_segments	Train/origin/destination segment for booking.	T4	segment_id; booking_id
seat_allocations	Maps passenger/booking to allocated seat/berth.	T4	allocation_id; seat_id
booking_status	Booking status/history records.	T4	booking_id + status/timestamp
cancellations	Cancellation reason, amount, timestamp and refund reference.	T4	cancellation_id; booking_id

T5 — PAYMENT

Table	Purpose	Owner	Key / Design Note
payments	Business-level payment associated with booking.	T5	payment_id; booking_id
payment_transactions	Gateway order/payment attempts and verification results.	T5	transaction_id; gateway IDs
refunds	Refund amount/status/gateway refund reference.	T5	refund_id; payment_id
payment_methods	Optional normalized payment method definitions.	T5	payment_method_id

T6 — PNR / LIVE

Table	Purpose	Owner	Key / Design Note
pnr_records	PNR lookup record and associated train/journey metadata.	T6	pnr_id; PNR UNIQUE

Table	Purpose	Owner	Key / Design Note
pnr_passengers	Passenger-wise PNR status, coach/berth and booking status.	T6	pnr_passenger_id; pnr_id
train_live_status	Latest live running state of a train.	T6	train_id; observed_at
train_locations	Location observations/coordinates for train tracking.	T6	train_id; observed_at
train_status_history	Historical status/delay/station events.	T6	train_id; event_time

T7 — ADMIN / AUDIT

Table	Purpose	Owner	Key / Design Note
audit_logs	Security/business audit events across modules.	T7	actor, action, entity, timestamp
api_request_logs	Optional API performance/error monitoring data.	T7	endpoint, status, latency, timestamp

CROSS-TEAM INTEGRATION CONTRACTS

Integration	Teams	Rule
Search → Train Details	T2 → T3	T2 returns stable train_id/train_number and station identifiers; T3 must not depend on raw external API response structure.
Availability → Booking	T3 → T4	T3 exposes current/demo availability; T4 re-checks availability transactionally before reserving.
Booking → Payment	T4 ↔ T5	Booking creates a payable reservation/order context; payment verification is server-side; booking is confirmed only after verified payment.
Payment → Booking	T5 → T4	Use a service/API contract rather than uncontrolled direct DB updates across modules.
Booking → PNR	T4 → T6	Booking generates/stores the demo PNR/reference used by the PNR module.
Train Master → Live Tracking	T2 → T6	T6 consumes stable train/station IDs and route data; live API data is normalized.
All Modules → Admin	T1–T6 → T7	T7 reads through approved APIs/views and does not bypass business rules for destructive operations.
All Teams → Database	All → DB	Schema changes require migration/SQL PR; no manual production schema edits.

GIT & PR WORKFLOW

```
main
■■■ team/auth
■■■ team/search
■■■ team/train-details
■■■ team/booking
■■■ team/payment
■■■ team/tracking
■■■ team/admin

Inside a team:
team/booking
■■■ feature/booking-ui
■■■ feature/booking-api
■■■ feature/booking-db
■■■ feature/booking-tests

feature branch → team branch → integration test → main
```

PR requirement: include screenshots for UI work, endpoint examples for backend work, migration/procedure/trigger files for DB work, and test evidence for all features. A team should not merge code that requires another team's unfinished branch unless the dependency is explicitly documented.

DATABASE RULES

• Target 3NF for transactional/master data unless a deliberate performance reason is documented.

• Every table needs a clear primary key; foreign keys should enforce real relationships.

• Use UNIQUE constraints for natural identifiers such as station code, train number, email and PNR where appropriate.

• Index foreign keys and frequent search/filter columns; verify expensive queries with EXPLAIN.

• Do not put secrets, passwords or gateway secret keys in database rows or frontend code.

• Stored procedures/triggers should have a clear purpose; avoid putting all business logic into triggers.

• All schema/procedure/trigger/view changes must be committed to Git and reproducible on a fresh database.

• Seed realistic demo data so every team can develop without depending on another team's local database.

FINAL END-TO-END OWNERSHIP

T1: Who is the user? → **T2:** Where can they go? → **T3:** Which train/class/seat is available? → **T4:** Can they book it? → **T5:** Can they pay? → **T6:** What is their PNR and where is the train? → **T7:** How is the whole system performing?